



BUILD & LEARN · DATA SCIENCE SATURDAYS

AI Engineering

WEEK 1

Psychology · Setup · Basic Code Structure · AI Tooling (CLI) · OpenAI Key

Saturday · 10:00–12:00 · Berlin

Hosted by Lindsey Peng

About This Course

A seven-week, hands-on program to build and deploy a real AI application — end to end. This is not theory-only machine learning: you'll ship working code touching FastAPI, Postgres, Docker, and a real AI pipeline.

WHO IT'S FOR

- Data scientists, analysts, and software developers
- Anyone transitioning into AI engineering
- Basic Python required — no prior LLM experience needed

WHAT YOU'LL TOUCH



FastAPI



Postgres



Docker



Real AI Pipeline

7 Weeks → **1 Deployed AI App in Your Own GitHub Repo**

Saturdays · 10:00–12:00 · Octopus Bar, Berlin

Week 1 Roadmap



Psychology

Mindset shift: from
practitioner to
orchestrator



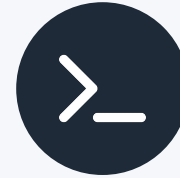
Setup

Environment: Python, Git,
Docker



Basic Code Structure

Folders, placeholders,
CLAUDE.md



AI Tooling (CLI)



OpenAI Key

Claude CLI workflow

API access & .env config

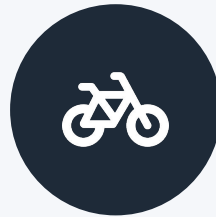
PART 1

Psychology



Why Psychology Comes First

Before we touch a single tool, we need to address the mental model. This course isn't about memorizing new syntax — it's about reshaping how you approach building software in the AI era.



Breaking Through the Fear

From practicing one skill to becoming an orchestrator of many

Breaking Through the Fear



"I haven't done this before."

Like learning to ride a bicycle or stand on a skateboard for the first time — the wobble is part of the process, not a sign you're doing it wrong.

This is a new era, and it requires a complete transition: from practicing a specific skill to becoming an orchestrator.

That means understanding, at a high level, how tools work together and how data travels between them — rather than the fine detail of how any single tool works internally.

PRACTITIONER

Knows one tool deeply



ORCHESTRATOR

Understands how the whole system fits together

Training You Two Ways

Becoming an AI engineer takes both reps and a real deliverable. This course trains you on two parallel tracks:



PRACTICING

Work WITH AI

Using AI in your daily workflow — including tools like Claude Code — until it becomes second nature.



BUILDING

An AI-Powered Service

Together, we build a real service where AI isn't bolted on — it's a core part of the workflow.

Work WITH AI



This isn't just a project to use AI as your main service — it's about working **WITH** AI in your daily workflow until it becomes second nature. That's exactly why we highly recommend Claude Code.

We start from nothing but an idea in our head — and ask AI to structure everything from there.

NOT THIS

AI as a tool you occasionally call for a task

INSTEAD

AI as your structuring partner, from first idea to working implementation

Optional but Highly Recommended

OPTIONAL · HIGHLY RECOMMENDED



Claude Pro — \$20/month

The Claude desktop app for writing, research, and problem-solving — plus Claude Code, a terminal-based AI coding agent that lets you delegate entire development tasks straight from your command line.



Claude Max — \$100 or \$200/month

For heavy users: 5–20x more usage than Pro, plus access to the most powerful Claude models.

This single subscription lets you ship faster, handle more complex projects, and operate like a small team. The ROI is a no-brainer.

An AI-Powered Service



This isn't a toy exercise — together, we build a real service where AI is a core part of the workflow, not a bolt-on feature.

By the end of this course, you'll have shipped an AI News API: scraping, enrichment, storage, and an API — all built together, step by step.

NOT THIS

A disposable tutorial project

INSTEAD

A real, working service — live in your own GitHub repo

PART 2

Actual Project Setup



Two Paths to Follow Along



Build From Scratch

Follow every step this week: install the tools yourself, scaffold the project by hand, and use Claude to help structure it from a blank idea.



Clone the Starter

Prefer to dive straight into working code? Clone the Week 1 branch of the course repo and follow along from there.

USING CLAUDE CODE?



```
$ git clone -b week-1 https://github.com/lindseypeng/ai-personal-assistant.git
```

[View the repo on GitHub](#)

What We're Building: An AI Personal Assistant

Here is the service we will build together, starting with Nylas email access and adding intelligence one workflow at a time:



Connects email and calendar accounts through Nylas



Receives real-time email events through signed webhooks



Normalizes provider payloads into typed application models

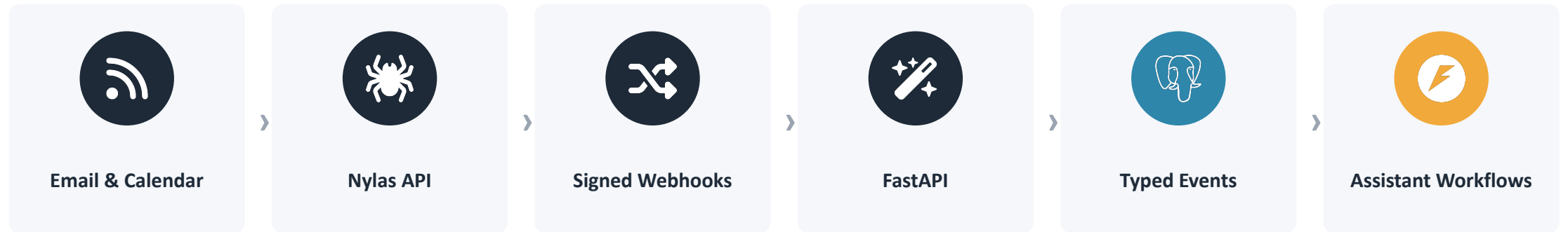


Uses AI workflows to classify requests and decide next steps



Takes approved email and calendar actions through Nylas

High-Level Architecture



Nylas sends provider events to FastAPI. The app verifies, models, displays, and routes each event into the assistant workflow.

Tech Stack



Python



Pydantic



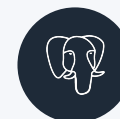
Nylas API



OpenAI API



SQLAlchemy



PostgreSQL



Docker



uv



Pinggy or Serveo



PROJECT SETUP

Design First — Draw Your Flow Diagram



Now that you've seen how the AI News API's pieces fit together, do the same for your own idea: before writing any code, sketch your project as an interactive flow diagram — map the pieces and how data moves between them.

Tip: tools like Excalidraw or FigJam work well for this — and Claude can help you talk through the design out loud.

1

PROJECT SETUP

Install Python & Git



Install Python 3.10+ and Git — the two foundational tools every project in this course builds on.



```
$ python3 --version  
$ git --version
```

PYTHON

- [Python Downloads Page](#)
- [Official Installation Guide](#)

GIT

- [Git Downloads Page](#)
- [Git Installation Guide](#)



PROJECT SETUP

RECOMMENDED TOOLING

Set Up Your IDE: Cursor

Cursor is a VS Code–based editor built for AI-assisted coding: intelligent autocomplete, refactoring suggestions, and contextual help are built directly into your workflow. Many engineers find that tools like this meaningfully speed up day-to-day development — it's worth a session to get comfortable with it.

SETUP

- [Cursor Download Page](#)
- [Video: How I Set Up VS Code for AI Projects](#)
- [Video: How to Work With Cursor](#)

2

PROJECT SETUP

Install Packages: uv & Docker



uv is a fast Python package and dependency manager. Docker lets us containerize the app so it runs the same way on every machine.

```
# install uv (see official docs for your OS)
$ curl -LsSf https://astral.sh/uv/install.sh | sh
# install Docker Desktop from docker.com
```

UV PACKAGE MANAGER

→ [Getting Started with uv](#)

3

PROJECT SETUP

OPTIONAL · RECOMMENDED

Install Claude CLI



The Claude CLI lets you work with Claude directly from your terminal — reading, writing, and running code alongside you.



```
$ claude --version  
# follow the official install guide for your OS
```



PROJECT SETUP

Give Claude Context: AGENT.md



Create a AGENT.md file with general context Claude should know about the project — background it needs, but nothing that requires immediate action.

Include: the project's goal, the tech stack, folder conventions, and your preferred coding style.

5

PROJECT SETUP

Create the Initial Project Structure



“Create the initial project structure exactly as defined in AGENT.md. Create the folders and placeholder files only. Do not implement business logic yet.”

```
project/  
├── src/  
├── tests/  
├── main.py  
└── AGENT.md
```

6

PROJECT SETUP

Hello World in main.py



Confirm the basic structure runs before adding anything else.

```
• • •
# main.py
def main():
    print("Hello, AI Engineer!")

if __name__ == "__main__":
    main()
```



PROJECT SETUP

Run uv sync



This creates your virtual environment and a uv.lock file — locking exact dependency versions so the project stays reproducible.



```
$ uv sync
```



PROJECT SETUP

Docker Compose & Docker Run



Start with the simplest possible setup: a Dockerfile that just copies your current code into the image.



```
$ docker compose up --build  
$ docker run your-image
```



PROJECT SETUP

GitHub Authentication & .env Setup



GitHub Authentication

Set up SSH or a personal access token so you can push your work. We'll walk through the steps together — or ask Claude/ChatGPT for a guided, step-by-step tutorial.



.env Setup

Copy the example environment file and fill in your own keys (including your OpenAI key).

```
$ cp .env.example .env
```

10

PROJECT SETUP

Understanding .env.example



If you cloned the repo instead of building from scratch, you'll see a `.env.example` file — a template listing which secrets and settings the project needs, without any real values filled in. Copy it, rename it to `.env`, and fill in your own keys there.

```

# .env.example - checked into the repo (safe to commit)
OPENAI_API_KEY=
DATABASE_URL=

# your local copy - never committed
$ cp .env.example .env
```

.env is already in .gitignore — only .env.example should ever be committed.

11

PROJECT SETUP

Set Up Your OpenAI API Key



You'll need an OpenAI account and API key to follow along with the exercises.

1. Visit platform.openai.com to create an account
2. Navigate to the API keys section in your account dashboard
3. Create a new secret key
4. Store it securely — we'll use environment variables to manage it

```
# .env
OPENAI_API_KEY=sk-...your-key-here...
```

GET YOUR KEY

→ platform.openai.com

PART 2

Additional Setup



11

PROJECT SETUP

Set Up Your Nylas Account: follow instruction [here](#)



You need a Nylas developer account so the assistant can connect to a development email account.

1. Create an account in the Nylas Dashboard
2. Create an application and choose its API region
3. Copy the API key and client ID
4. Add `http://localhost:5010/oauth/exchange` as a callback URL

```
# .env
NYLAS_API_KEY=your-api-key
NYLAS_CLIENT_ID=your-client-id
```

NYLAS DASHBOARD → dashboard-v3.nylas.com

WEEK 1 — RECAP

You're Set Up — What's Next

- ✓ Made the mindset shift: practitioner → orchestrator
- ✓ Explored the AI Personal Assistant — the project we're building
- ✓ Installed Python, Git, uv, and Docker
- ✓ Scaffolded the project structure from AGENT.md
- ✓ Ran Hello World, uv sync, and docker compose
- ✓ Connected GitHub and set up your .env
- ✓ Got your OpenAI API key configured

NEXT SATURDAY

Week 2: The LLM Layer

Connecting your first LLM layer and starting the FastAPI backend.

Bring your laptop, your curiosity, and your AGENT.md.